LABORATORIO #1 · SOLUCIONES DE NEGOCIO: POO + n8n

Panel de riesgo de proveedores con comprobante QR

Estudiante: Victor Gonzalez **Fecha:** 19 de octubre de 2026 **Repositorio:** github.com/victorgonzalez246/Panel-de-riesgo-de-proveedores-con-comprobante-QR

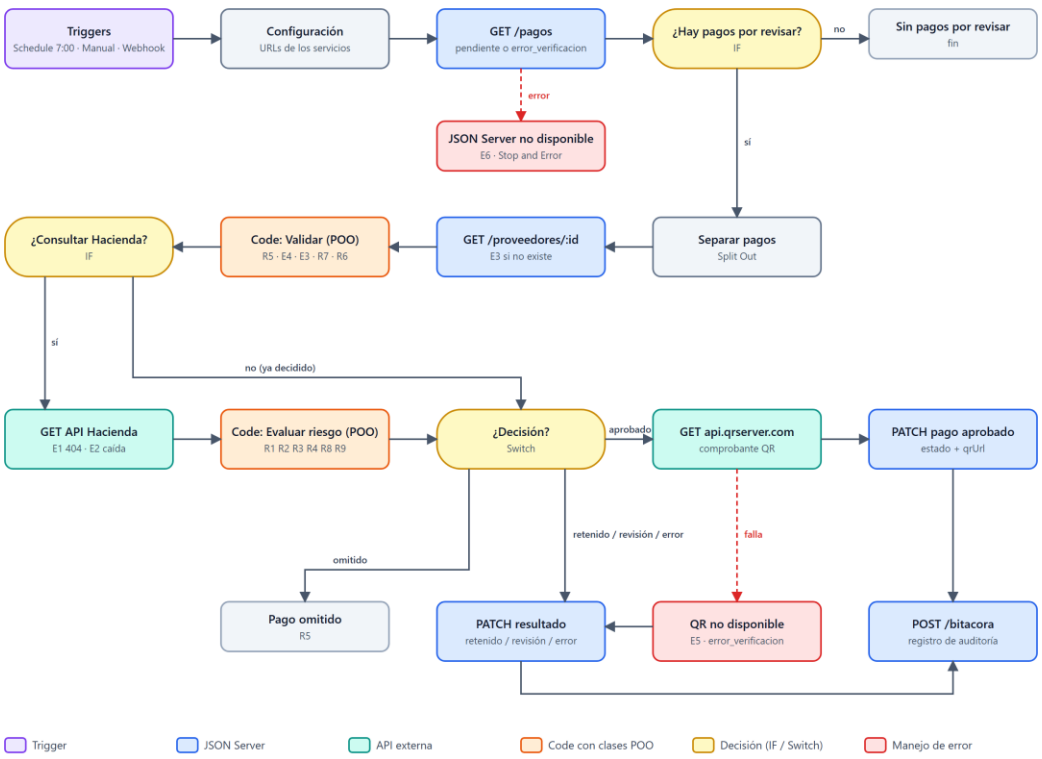
1. El problema y por qué se beneficia de esta solución

Antes de pagarle a un proveedor, una empresa costarricense debe asegurarse de que ese proveedor esté al día con el Ministerio de Hacienda. Pagarle a un contribuyente moroso u omiso expone a la empresa a observaciones de auditoría y a problemas para deducir el gasto. En la práctica, esta revisión la hace una persona de tesorería entrando al sitio de Hacienda, cédula por cédula, y anotando el resultado en una hoja de cálculo. Es lento, se olvida con facilidad y no deja rastro de quién revisó qué y cuándo.

La solución automatiza esa revisión: todos los días a las 7:00 a. m. un workflow de n8n toma los pagos pendientes registrados en JSON Server, valida los datos, consulta la situación tributaria real del proveedor en la API pública de Hacienda y decide. Si el proveedor está al día, el pago se aprueba y se genera un **comprobante QR** con los datos del pago (api.qrserver.com). Si está moroso, omiso o hay algo raro, el pago queda **retenido** o en **revisión manual** con un motivo claro. Cada decisión queda registrada en una bitácora. Así tesorería solo atiende los casos que de verdad necesitan una persona.

2. Diseño del workflow

El workflow tiene 28 nodos (incluida una nota con el resumen de reglas). El diagrama muestra el recorrido principal de los datos (sin el historial y el aviso por correo de la sección 7); los colores indican el tipo de nodo:



Recorrido. Cualquiera de los tres triggers (Schedule diario a las 7:00, Manual, o el Webhook que usa el panel web) pasa por **Configuración**, que guarda las URL de los tres servicios para poder cambiarlas en un solo lugar. Luego se piden a JSON Server los pagos en estado `pendiente` o `error_verificacion`. Un IF corta el flujo si no hay nada que revisar; si JSON Server no responde, una rama de error detiene la ejecución con un mensaje claro (E6). Los pagos se separan en ítems y, por cada uno, se busca su proveedor. El primer nodo Code (**Validar**) aplica las reglas que no necesitan a Hacienda; el IF **¿Consultar Hacienda?** solo deja pasar a Hacienda los pagos válidos, para no gastar consultas en datos malos. El segundo nodo Code (**Evaluar riesgo**) interpreta la respuesta de Hacienda y decide. Por último, un Switch reparte los resultados: los aprobados generan su QR y se actualizan con `estado` y `qrUrl`; el resto se actualiza con su estado y motivo; todos terminan en la bitácora.

3. Reglas de negocio

Implementé nueve reglas (el mínimo era cinco). Cada una tiene un código que aparece en el texto del motivo guardado en JSON Server, así se puede rastrear desde el registro hasta el método que la aplicó.

Regla	Descripción	Método de la clase	Dónde se ve en el workflow
R1	Proveedor moroso ante Hacienda → retenido	`Proveedor.estaMoroso()`	Code Evaluar → PATCH resultado
R2	Proveedor omiso → retenido	`Proveedor.estaOmiso()`	Code Evaluar → PATCH resultado
R3	Inscripción distinta de "Inscrito" o "Inscrito de oficio" → retenido	`Proveedor.estaInscrito()`	Code Evaluar → PATCH resultado
R4	Monto mayor a ₡5 000 000 o $10 000 → revisión manual, aunque esté al día	`Pago.requiereAprobacionManual()`	Code Evaluar → PATCH resultado
R5	Solo se revisan pagos pendientes o con verificación fallida	`Pago.esProcesable()`	Filtro del GET /pagos y Code Validar
R6	Cédula con formato inválido → retenido sin consultar Hacienda	`Proveedor.validarCedula()`	Code Validar → IF (rama "no")
R7	Proveedor inactivo en el registro de la empresa → retenido	`Proveedor.estaActivo()`	Code Validar → IF (rama "no")
R8	Pago vencido → prioridad alta	`Pago.estaVencido()`, `prioridad()`	Campo `prioridad` de cada pago
R9	Al tercer intento fallido de verificación → revisión manual	`Pago.registrarFallaVerificacion()`	Campo `intentosVerificacion`

Situaciones inesperadas. El workflow contempla seis, y ninguna aprueba un pago por defecto: ante la duda, el pago queda detenido.

Caso	Situación	Cómo se maneja
E1	La cédula no existe en Hacienda (HTTP 404)	Pago retenido: el proveedor no está registrado como contribuyente.
E2	Hacienda no responde, responde 5xx/429 o sin la situación tributaria	Estado `error_verificacion`; se reintenta en la siguiente corrida (R9 limita a 3).
E3	El proveedor del pago no existe en JSON Server	Revisión manual: hay un error en los datos internos.
E4	El pago llega incompleto (sin monto, moneda o proveedor)	Revisión manual indicando qué campo falta.
E5	api.qrserver.com no genera el comprobante	El nodo reintenta 3 veces; si sigue fallando, `error_verificacion`.
E6	JSON Server está apagado	Stop and Error con el mensaje de cómo levantarlo.

4. Cómo se conectan las clases con el nodo Code

Modelé el escenario con clases en JavaScript puro, cada una con una sola responsabilidad (`DiagnosticoProveedor` reutiliza estas mismas reglas para la consulta del panel web):

- `Proveedor`: sabe si su cédula es válida, si está activo y, una vez que recibe la respuesta de Hacienda, si está moroso, omiso o mal inscrito (`impedimentosTributarios()` junta R1, R2 y R3).
- `Pago`: valida sus propios datos, sabe si requiere aprobación manual o si está vencido, y controla sus cambios de estado (`aprobar`, `retener`, `enviarARevision`, `registrarFallaVerificacion`). Cada transición devuelve exactamente el objeto que se envía en el PATCH a JSON Server.
- `ComprobanteQR`: arma el código del comprobante (`CMP-PAG-001-20261019`), el texto que va dentro del QR y la URL de api.qrserver.com.
- `ReglaNegocio` y sus subclases: cada regla (R1, R2, R3, R4, R6, R7) es una clase que hereda de `ReglaRetencion` o `ReglaRevisionManual`, y estas de `ReglaNegocio`.
- `EvaluadorRiesgo`: coordina a las demás. Tiene dos métodos que corresponden uno a uno con los dos nodos Code: `validar()` (antes de Hacienda) y `evaluar()` (después de Hacienda).

Para cumplir la regla de "una sola lógica", el script `scripts/build-workflow.js` copia el contenido de los archivos de `src/` dentro de los dos nodos Code al generar el JSON del workflow. El nodo **Validar** termina con `evaluador.validar(pago, $json)` y el nodo **Evaluar riesgo** con `evaluador.evaluar(previo.pago, previo.proveedor, $json)`. Es el mismo código que ejecuta `main.js` fuera de n8n, así que si cambio un umbral en `src/config.js` y regenero el workflow, el cambio aparece en ambos lados. Lo comprobé ejecutando las dos cosas con los mismos datos: los diez pagos de prueba terminan exactamente en el mismo estado y con el mismo motivo.

Los cuatro pilares de POO. La jerarquía de reglas es donde mejor se ven:

Pilar	Cómo se aplica en el proyecto
Abstracción	`ReglaNegocio`, `ReglaRetencion` y `ReglaRevisionManual` son abstractas: lanzan un error si se intentan instanciar y obligan a cada regla a implementar `aplica()` y `motivo()`.
Herencia	`ReglaMoroso`, `ReglaOmiso`, `ReglaNoInscrito`, `ReglaCedulaInvalida` y `ReglaProveedorInactivo` heredan de `ReglaRetencion` (decisión "retenido"); `ReglaMontoAlto` hereda de `ReglaRevisionManual`. Todas reciben el método `evaluar()` de `ReglaNegocio`.
Polimorfismo	`EvaluadorRiesgo` recorre una lista de reglas y llama `regla.evaluar(contexto)` sin saber cuál es. Para agregar una regla basta con crear otra subclase y sumarla a la lista; una prueba lo demuestra con una regla R10 inventada.
Encapsulamiento	`Pago` solo cambia de estado mediante sus métodos públicos (`#cambiarEstado` es privado) y `Proveedor` interpreta su propia situación tributaria.

5. Decisiones técnicas

- **Dos nodos Code en lugar de uno.** Validar antes de llamar a Hacienda evita gastar consultas en pagos que igual se iban a rechazar (cédula inválida, proveedor inactivo o inexistente). La API de Hacienda limita las consultas seguidas, por eso además se consulta de una cédula a la vez con 1,5 s de pausa.
- **"Full Response" y "Continue On Fail" en los HTTP.** Así un 404 o una caída no detienen el workflow: llegan como datos (`statusCode` o `error`) a las clases, que deciden qué significan. La lógica del error vive en la clase, no escondida en la configuración del nodo.
- **Nunca aprobar por defecto.** Cualquier falla deja el pago retenido, en revisión o en error para reintento. Un error temporal (E2, E5) se reintenta solo hasta tres veces; después pasa a una persona (R9).

- **Estado intermedio error_verificacion.** Diferencia "Hacienda está caída" (temporal) de "el proveedor tiene un problema" (definitivo), y permite reintentar sin intervención.
- **JSON Server sin --watch y escrituras de una en una.** En las pruebas descubrí que, en Windows, JSON Server se cae (error EPERM) si recibe varias escrituras simultáneas, y que con `--watch` se recarga a sí mismo tras cada cambio. Las escrituras se envían en lotes de uno con 150 ms de pausa. También uso `127.0.0.1` en vez de `localhost`, porque n8n resolvía `localhost` como IPv6 y JSON Server escucha en IPv4.
- **Datos reales.** Los proveedores de prueba usan cédulas jurídicas reales: el ICE aparece moroso, la CCSS aparece "Inscrito de oficio" (por eso R3 acepta ese estado) y una cédula con formato válido devuelve 404. Los casos omiso y desinscrito no los encontré en datos reales, así que los cubren las pruebas automatizadas con respuestas simuladas.
- **Agregado opcional: panel web.** Una página servida por JSON Server permite registrar pagos y proveedores (validados con las mismas clases), consultar una cédula, lanzar la revisión y ver los comprobantes QR y los avisos por correo. No tiene lógica propia: llama a webhooks de n8n que usan las mismas clases (`DiagnosticoProveedor` para la consulta).

6. Pruebas realizadas

`npm test` ejecuta 35 pruebas automatizadas que cubren las reglas R1 a R9, los errores E1 a E4, la consulta de cédula del panel, los pilares de POO, el correo de resumen y la tendencia (E5 y E6 dependen de servicios caídos y se probaron directamente en n8n). `node main.js` muestra los métodos de cada clase y evalúa los diez pagos (con `--en-vivo` consulta Hacienda en el momento). El workflow lo ejecuté en n8n 2.38 en cinco escenarios: corrida normal (2 aprobados, 4 retenidos y 3 en revisión; PAG-010, que ya estaba aprobado, ni siquiera se lee), segunda corrida sin pendientes, servicio QR caído y luego recuperado, Hacienda caída tres veces seguidas (hasta llegar a R9) y JSON Server apagado. En todos los casos el estado final en JSON Server coincidió con lo esperado. Además, GitHub Actions corre las pruebas y `main.js` en cada cambio que se sube al repositorio, y verifica que los workflows sigan generados a partir de las clases.

7. Extensión del laboratorio

- **Reto 1 · Herencia y polimorfismo.** La jerarquía `ReglaNegocio` → `ReglaRetencion` / `ReglaRevisionManual` → seis reglas concretas que redefinen `aplica()` y `motivo()` (ver sección 4). Los nodos Code de n8n usan esa misma jerarquía.
- **Reto 3 · Notificación por correo.** Al terminar cada revisión, la clase `ResumenRevision` arma un único correo para tesorería con los proveedores morosos destacados, cada pago retenido o en revisión con su motivo y los QR de los aprobados, y solo se envía si hay situaciones críticas. Como las ramas del workflow llegan a la bitácora por separado, el nodo **Preparar notificación (POO)** espera a que estén registrados todos los pagos (cuenta las corridas con `$runIndex`) para enviar un solo correo y no tres. Lo probé con un servidor SMTP local; para el envío real usa una cuenta de Gmail por SMTP.
- **Reto 4 · Historial y tendencia.** Cada revisión queda guardada en la colección `revisiones` de JSON Server. Al inicio, el workflow lee las últimas cinco; al final, la clase `HistorialTendencia` calcula el índice de riesgo (pagos detenidos o en revisión sobre pagos revisados) y lo compara con el promedio anterior: **empeorando** o **mejorando** si cambia más de 10 puntos, **estable** si no, o **sin historial** en la primera revisión. La conclusión va en el correo, en la revisión guardada y en el panel.